Лекция 02 — Обработка ошибок

Содержание

1. Общие представления
    1. Постановка проблемы
    2. `assert`
    3. Коды возврата
2. Исключения
    1. Процесс обработки исключений
    2. Синтаксис
    3. Типы исключений
    4. Гарантии исключений
    5. Управление выбросами
    6. О хорошем и плохом тоне
3. Техническая реализация исключений
    1. Контекст
    2. Пробрасывание исключения
    3. Отлов исключения
    4. Проблемы, проблемы, проблемы
4. `std::expected` (C++23)
    1. Мотивация
    2. Интерфейс
    3. Монадические операции
5. N способов убить программу

> *”Если Яндекс ломается, то все расстраиваются… Ну, не все, но я точно расстраиваюсь.”*
> — А. П. Хвастунов

Общие представления

Постановка проблемы

Мы живем в жестоком мире, где количество внешних обстоятельств, которые способны (и охотно это сделают) нарушить ход выполнения нашей программы, тяжело оценить даже самыми большими кардиналами. Иными словами, ошибок бывает много. Они возникают

повсеместно, и уж никто, конечно, не любит, когда программа не может нормально выполняться. По этой причине любые исключительные ситуации всегда следует диагностировать и обрабатывать.

В вопросе реализации этого процесса хватает простора для воображения. Как это часто случается, универсального решения у этой задачи нет: разные подходы обладают своими преимуществами и недостатками и используются в зависимости от конкретной ситуации. В современном C++ чаще всего используется механика исключений, которая будет рассмотрена далее, но перед этим следует упомянуть несколько менее часто встречаемых методов, которые, впрочем, составляют необходимый контекст или даже бывают полезны сами по себе.

`assert`

`assert` — это определенный в заголовочном файле `<cassert>` функциональный макрос языка Си, который динамически (т.е. в процессе выполнения программы) проверяет валидность переданного ему аргумента скалярного типа и вызывает `std::abort` в случае, если он равен нулю:

```
int main() {
    assert(2 + 2 == 5);
}

// int main(): Assertion `2+2==5' failed.
// Aborted (core dumped)
```

Как можно видеть, в момент аварийного завершения процесса `assert` также выводит в консоль справочную информацию об ошибке. Если же мы хотим реализовать всю ту же семантику на этапе компиляции, следует использовать ключевое слово `static_assert`.

Он служит простейшим методом диагностики ошибок, однако предел его способностей — завершить программу, что далеко не всегда нас устраивает. Он имеет место как вспомогательный инструмент при разработке, однако его ни в коем случае не стоит оставлять в финальных сборках — из этих соображений существует макрос `NDEBUG`, который полностью отключает все проверки внутри `assert`.

Коды возврата

Итак, `assert` помогает нам с диагностикой ошибок, но совершенно не помогает с их обработкой, ибо при возникновении каких-либо неординарных ситуаций мы далеко не всегда хотим, чтобы программа падала. Главные трудности возникают при попытке проверить корректность выполнения сторонних функций, потому что мы далеко не всегда можем обработать ошибку в рамках того модуля, в котором она произошла, а если и

можем, то хорошо бы уведомить обратившуюся к нам часть программы о том, что случилось страшное и что результатам наших вычислений лучше не доверять.

На этом этапе мы сталкиваемся с повсеместно использующейся в Си и типовой для мира процедурного программирования идеей — *кодами возврата*. Практически она состоит в том, чтобы любым способом передать сообщение об ошибке обработчику, который уже будет ответственен за поддержание нормального хода выполнения программы в самых страшных ситуациях. Мы можем использовать код возврата в качестве возвращаемого значения, можем записывать его в один из специально переданных аргументов, а можем обновлять глобальные переменные[1]. Все из этих подходов встречаются в стандартной библиотеке. Вот так, например, выглядит безопасная запись в файл с использованием кодов возврата:

```
int main() {
    std::FILE* out = std::fopen("output.txt", "w");
    if (out == nullptr) {
        // handle
    }
    if (std::fputs("Hello, World!\n", out) == EOF) {
        // handle;
    }
    if (std::fclose(out) == EOF) {
        // handle;
    }
}
```

Это отлично работает и защищает нас по всем фронтам, однако имеет один значительный недостаток. С точки зрения бизнес-логики здесь происходит ровно три действия: открытие файла, запись в файл, закрытие файла. Тем не менее, строк кода здесь заметно больше трех, и это главная проблема кодов возврата: их обработка порождает огромное количество технического кода, за которым совершенно теряется содержательная часть программы. А ещё про обработку кодов возврата очень легко забыть.
Нам очень хотелось бы сохранить их семантику так, чтобы не разменивать при этом безопасность на лаконичность, и ровно для этого в язык C++ введена механика исключений.

Исключения

Процесс обработки исключений

Механизм обработки исключений (*exception handling*) предоставляет простой и лаконичный интерфейс для передачи управления и данных ассоциированной подпрограмме-обработчику.

Когда в программе возникает внештатная ситуация, мы создаём объект-исключение, который сигнализирует компилятору о том, что нормальное продолжение работы невозможно, после чего он начинает поиск обработчиков этого исключения. Если в текущем фрейме стека таковой находится, то программа продолжает выполняться с его тела. В противном случае информация об исключении передаётся выше по стеку, сообщая вызвавшей нас части программы о возникновении необработанной исключительной ситуации, и процесс повторяется. В случае, если ни одна функция не оказывается в состоянии обработать исключение, вызывается обработчик по умолчанию, которым является `std::terminate` (про него мы ещё скажем отдельно) — иными словами, программа аварийно завершается.

В процессе подъёма по стеку в поисках подходящего обработчика все промежуточные фреймы очищаются путём вызова деструкторов у всех объектов с автоматическим временем жизни. Это делается из того соображения, что функции, не обработавшие ошибку, не могут или, во всяком случае, не должны продолжать своё выполнение, поскольку программные данные для них находятся в невалидном состоянии.

Весь этот процесс называется *раскруткой стека* (*stack unwinding*). Она является ключевой механикой, стоящей за исключениями.

Если в процессе раскрутки стека деструктор какого-то из объектов (или ещё кто-либо) выбросил исключение, компилятор сразу вызывает `std::terminate`, поскольку ситуация с существованием в системе более чем одного исключения в один момент времени не поддаётся адекватной обработке.

Синтаксис

Взаимодействие с интерфейсом происходит при помощи трёх основных конструкций.

> 1. `throw`

Исключение порождается путём *копирующей* инициализации на основе результата вычисления специального `throw`-выражения:

```
int main() {
    throw "I'm an exception!";
}
```

В качестве исключения может выступать любой объект, кроме:

- объектов неполного типа и указателей на них (кроме указателей на `void`);
- экземпляров абстрактного класса.

> 2. `try` / `catch`

Блок кода, потенциально порождающий исключение, оборачивается в фигурные скобки, сопровождаемые оператором `try`, что позволяет компилятору связать с фрагментом, в котором возникает ошибка, конкретный набор обработчиков, находящихся в аналогично структурированных `catch`-блоках.

Общий вид конструкции `try`-`catch` выглядит следующим образом:

```
try {
    // код, выбрасывающий исключение
}
catch(/* тип исключения */) {
    // тело обработчика
}
```

При объявлении `catch`-блока в круглых скобках указывается тип исключения, которое он обрабатывает. Это может быть именованный параметр, неименованный параметр (то есть просто имя типа), а также многоточие, которое задает пользовательский обработчик по умолчанию, способный принимать любой тип исключений:

```
try {
    // ...
}
catch(int& exc) {
    // обработает исключение типа int с доступом к его значению
}
catch(float&) {
    // обработает исключение типа float без доступа к значению
}
catch(...) {
    // обработает любое исключение
}
```

Во избежание ненужного копирования и слайсинга рекомендуется принимать значения в обработчиках по ссылке.

В процессе раскрутки стека компилятор просматривает все обработчики в той последовательности, в которой они встречаются в коде. Это важно, поскольку неудачный

порядок объявления `catch`-блоков может привести к тому, что исключение будет обработано некорректным образом:

```
class A {};
class B : public A {};

int main() {
     try {
         throw B();
     }
     catch(A&) {
         std::cout << "A&";
     }
     catch(B&) {
         std::cout << "B&";
     }
}
```

Об этом, впрочем, обычно уведомляет компилятор, выдавая варнинг. Обработчик по умолчанию компилятор вообще не даст описать раньше какого-либо другого.

Компилятор проверяет типы выброшенного программой и принимаемого обработчиком исключений на предмет полного совпадения, возможности приведения, участия в иерархии наследования и ковариантности — словом, всё кажется достаточно понятным и прозрачным. Хотя в действительности только кажется, мы опустим этот момент и поверим, что всё действительно хорошо. [2]

Пойманные исключения также можно *перебрасывать*, то есть форсировать их продвижение по стеку уже после того, как они были пойманы одним из обработчиков. Для этого следует использовать пустое `throw`-выражение:

```
void foo() {
     try {
          throw 238;
     }
     catch(int) {
          std::cout << "foo(): catched!\n";
          throw;
     }
}

int main() {
     try {
          foo();
```

```
    }
    catch(int) {
        std::cout << "main(): catched!\n";
    }
}
```

Примечательно, что пустой `throw` передаёт дальше именно исходное исключение, а не конструирует новое из аргументов `catch`-блока.

Стоит также упомянуть, что в `try`-блоки можно оборачивать тела функций, в том числе специальных методов класса. Это, в частности, позволяет обезопасить вызов делегирующих конструкторов:

```
struct A {
    A() {
        throw 238;
    }
};

struct B {
    A a;
    B() try : a() {}
    catch(...) {
        std::cout << "Handled!\n";
    }
};
```

Очень важно, что в конструкторах и деструкторах исключения *всегда* автоматически перебрасываются, поскольку иначе объект остался бы лишь частично сконструирован или уничтожен. Из забавного: `main` тоже можно обернуть в `try`-`catch`, и он будет отлавливать почти все исключения, кроме возникающих в специальных методах статических объектов.

Отметим отдельно, что на исключение можно получить указатель специального типа `std::exception_ptr`, который семантически является чем-то вроде `std::shared_ptr`. Это позволяет отвязать исключение от стека и времени, превращая его в объект, которым может управлять сам пользователь. Указатели на исключения находят особое применение при работе с многопоточными и асинхронными вычислениями, что, конечно, очень здорово, но вдаваться мы в это не будем.

Типы исключений

Хотя исключением может являться практически что угодно, использование стандартных типов в этой ситуации является очевидно плохим решением, поскольку они не несут

совершенно никакой контекстуально полезной нагрузки.

Из этих соображений был введён специальный класс `std::exception`, который предоставляет единообразный интерфейс для передачи информации о характере исключения. Он обладает необходимым рядом специальных методов и единственным виртуальным методом `what`, который возвращает константную си-строку с содержанием брошенного исключения.

Все исключения в стандартной библиотеке наследуются от `std::exception`. В ней представлено широкое разнообразие таких дочерних классов, которые соответствуют различным видам ошибок. Некоторые из них позволяют пользователю самостоятельно определять, какое сообщение вернёт метод `what`:

```
int main() {
    try {
        throw std::runtime_error("My message");
    }
    catch(std::runtime_error exc) {
        std::cout << exc.what(); // My message
    }
}
```

Мы тоже можем наследоваться от `std::exception`, и будет очень даже хорошо, если мы действительно будем, поскольку это позволяет поддерживать единообразие и понятность кода.

Гарантии исключений

Помимо того, что мы можем сообщить программе о возникновени исключения, мы также можем дать дополнительные гарантии касаемо того, что будет служить результатом его обработки. Выделяется четыре категории таких гарантий:

> I. Nothrow (nofail) гарантия
>
> Функция не выбрасывает исключений.
>
> *Nothrow* гарантия подразумевает, что ошибки обрабатываются любым другим способом, или же о них не сообщается. Она ожидается, в частности, от деструкторов и других функций, которые могут быть вызваны в процессе раскрутки стека.
>
> *Nofail* гарантия подразумевает, что функция всегда работает корректно. Она ожидается от функций `swap`, конструкторов перемещения и других функций, которые используются функциями со строгой гарантией исключений.

> II. Строгая гарантия
>
> Если функция выбрасывает исключение, состояние программы откатывается до того, в котором находилась до его возникновения. Из соображений соблюдения этой гарантии существует, например, рассмотренная ранее идиома Copy-and-Swap.

> III. Базовая гарантия
>
> Если функция выбрасывает исключение, состояние программы остаётся корректным: никакие ресурсы не утекли, и инварианты всех объектов сохранились.

> IV. Отсутствие гарантии
>
> Если функция выбрасывает исключение, нет гарантии, что состояние программы останется корректным. Возможно, умрёт чья-то домашняя рыбка.

Управление выбросами

Язык предоставляет возможность закрепить nothrow гарантию в виде контракта для компилятора путём указания спецификатора `noexcept`, который полностью отключает генерацию дополнительного кода для обработки исключений. Он не запрещает выброс исключений из тела функции, однако полностью лишает компилятор возможности их обработать, поэтому любое исключение приводит к вызову `std::terminate`:

```
void foo() noexcept {
    throw 1;
}

int main() {
    try {
        foo();
    }
    catch(...) {
        std::cout << "Handled!\n";
    }
}

// terminate called after throwing an instance of 'int'
```

Примечательно, что `noexcept` может также выступать в роли оператора `noexcept(condition)`, который даёт эту гарантию лишь при выполнении внутреннего условия:

```
void foo() noexcept(false) {
    throw 1;
}

int main() {
    try {
        foo();
    }
    catch(...) {
        std::cout << "Handled!\n";
    }
}

// Handled!
```

Начиная с C++11, деструктор по умолчанию помечен `noexcept`, ибо не нужно выбрасывать исключения из деструктора.

До C++17 существовала возможность динамической спецификации при помощи `throw()`, которая ограничивала диапазон выбрасываемых значений рядом конкретно заданных типов:

```
void foo() throw(float) {
    throw 1;
    // throw 1.f; — всё сработает!
}

int main() {
    try {
        foo();
    }
    catch(...) {
        std::cout << "Handled!\n";
    }
}

// terminate called after throwing an instance of 'int'
```

Тем не менее, она была исключена из стандарта из-за огромного оверхеда и, что уж там, откровенной бесполезности: это совершенно никак не помогало оптимизациям, а лишь увеличивало размер бинарника.

Можно также упомянуть тип `std::nothrow_t`, который позволяет обращаться к неброcающим перегрузкам стандартных функций — в частности, операторов `new` и

`delete`, которые при передачи константы `std::nothrow` вместо исключения `bad_alloc` будут возвращать `nullptr`:

```
int main() {
    int* x = new(std::nothrow) int[10000000000ul];
    std::cout << std::boolalpha << (x == nullptr); // true
}
```

О хорошем и плохом тоне

Исключения — это сложно.
Они сложны, поскольку ломают наше представление о нормальном ходе выполнения программы, который обычно линеен с поправкой на `n` ответвлений условного оператора. И даже если мы понимаем, как с ними жить, то вот наша система, к сожалению, всегда будет от них страдать — этому посвящён следующий раздел.

По этой причине критически важно с умом подходить к использованию исключений и ни в коем случае не делать их частью нормального хода выполнения программы. Исключения возникают в *исключительных* ситуациях. Они всегда должны быть обоснованы и понятны.

Не следует использовать исключения для обработки событий. Не следует злоупотреблять стандартными обработчиками. Не следует бросать числа или строки. Не следует злоупотреблять `noexcept`. Ни в коем случае не следует помечать деструкторы `noexcept(false)`. Не следует принимать исключения по значению.
Пишите хороший код и живите счастливо.

Техническая реализация исключений

В этом разделе мы кратко обсудим, как реализована механика исключений с точки зрения современных компиляторов. Мы не будем вдаваться в технические подробности[3], однако некоторое понимание процесса необходимо для дальнейшего разговора о проблемах, которые возникают при работе с исключениями и об альтернативах, к которым мы можем прибегнуть.

Контекст

Прежде всего стоит упомянуть, что на низком уровне исключения можно, конечно, реализовывать по-разному.

Здесь следует выделить два основных подхода. Первый заключается в минимизации издержек в тех случаях, когда исключения не возникают, второй — в размене ненулевых пассивных затрат на более быструю раскрутку стека. Второй подход, в частности, реализовывался некоторыми старыми версиями компилятора MSVC, однако сейчас практически все компиляторы остановились на первом варианте, поскольку в том, чтобы пассивно нагружать всю систему ради обработки единственного исключительного случая из миллиона, достаточно мало смысла.

Старые версии GCC могли использовать метод SJLJ (от названия функций `setjmp` / `longjmp` в Си) для реализации перемещения по стеку, что было крайне медленно и, в сущности, вобрало в себя всё худшее из обоих подходов, поэтому от этого чуда быстро отказались в пользу табличного подхода на основе отладочной информации в формате DWARF.

Бинарный интерфейс обработки исключений, который реализуют, в частности, современные версии GCC и clang, подробно описан в Itanium C++ ABI, поэтому на этой документации мы и остановимся.

Пробрасывание исключения

Механизм порождения исключения достаточно прост. Любой `throw` преобразуется в следующие вызовы:

> 1. `__cxa_allocate_exception` / `__cxa_free_exception`

Функции с достаточно говорящими названиями, которые отвечают за выделение и освобождение памяти под объект-исключение. Обычно исключения выделяются на куче, однако поскольку нам также нужно где-то хранить исключение, которое сообщает об отсутствии места в этом сегменте памяти, оно также может быть выделено в специальном (скорее всего, статическом) экстренном буфере.

Само исключение формально представляет из себя пару из заголовка типа `__cxa_exception`, содержащего техническую информацию, необходимую для раскрутки стека, и непосредственно выбрасываемого объекта.

> 2. `__cxa_throw`

Функция, реализующая всю магию пробрасывания. Она ответственна в первую очередь за вызов раскрутки стека, то есть за передачу заголовка исключения в системную библиотеку `libunwind`.

Отлов исключения

С отловом исключений дела обстоят несколько сложнее.

В процессе раскрутки стека компилятор анализирует определённую для каждого фрейма CFI, — *Call Frame Information*, — чтобы узнать, может ли он обработать исключение, и если да, то с какого `catch`-блока следует продолжить выполнение программы. Этим действом управляет некто *персональная функция* (*Personality routine*), которая служит интерфейсом между библиотекой раскрутки и семантикой языка. Она работает с разделом LSDA, — *Language Specific Data Area*, — указатель на который располагается в CFI и который содержит всю информацию, связанную с специфичной для языка семантикой обработки исключений.

Работа персональной функции разбита на две фазы. В первой фазе она анализирует стек в поисках подходящего обработчика, во второй — возможно, вызывает функции очистки промежуточных фреймов (описанные в LSDA) и передаёт управление обработчику. Напомним, что обработчиком по умолчанию является `std::terminate`, и при его вызове стек обычно не очищается. Это важный момент, поскольку именно наличие двух фаз позволяет сохранить трассировку стека даже при отсутствии подходящего обработчика.

Для определения типа обрабатываемого исключения и поиска подходящего `catch`-блока используется уже знакомый нам из разговора о виртуальных таблицах механизм RTTI — в LSDA каждого фрейма хранятся поля `type_info` заголовков поддерживаемых исключений. Персональная функция анализирует эту информацию, проверяя тип брошенного исключения на точное совпадение, наследование или ковариантность каждому из типов в LSDA.

Проблемы, проблемы, проблемы

Мы уже обсуждали, что даже на уровне интерфейса исключения — не самая тривиальная в обращении механика. Теперь мы вдовесок знаем, что их обработка представляет собой *очень* ресурсоёмкую задачу для компилятора, которая связана с вполне ощутимыми накладными расходами. Об этих расходах, впрочем, следует упомянуть отдельно.

Современная модель исключений ещё называется zero cost в том смысле, что она, как было упомянуто, практически не нагружает систему, в которой их не возникает. Это, впрочем, касается производительности; что до размера бинарного исполняемого файла, то оверхед от хранения вспомогательной информации (которая создаётся всегда, если не указан спецификатор `noexcept` или флаг `-fno-exceptions`) может достигать 15-30%, хотя это сильно варьируется от характера и объёмов кода. Для высокоуровневых приложений это, впрочем, скорее не критично.

А вот когда исключение всё-таки выброшено, всё становится совсем весело. Помните ведь, что исключения лучше не использовать для обработки событий? Возьмём следующий простейший код для вычисления суммы модулей элементов массива:

```
int sum = 0;
for (int x : a) {
    if(x < 0) {
        sum += -x;
    } else {
        sum += x;
    }
}
```

А теперь допустим, что кто-то не очень добросовестный решил реализовать ту же семантику при помощи исключений:

```
int sum = 0;
for (int x : a) {
    try {
        sum += get_positive_value(x);
    } catch (...) {
        sum += -x;
    }
}
```

Так вот, бенчмарки могут показывать замедление производительности второго кода вплоть до десяти тысяч раз.

Идём дальше! Ещё один совершенно примитивный код:

```
struct invalid_value {};

void do_sqrt(std::span<double> values) {
    for (auto& v: values) {
        if (v < 0) throw invalid_value();
        v = std::sqrt(v);
    }
}
```

Если на AMD EPYC 7713 вызывать эту функцию на массиве из 100 элементов типа `double`, среди которых с вероятностью 0.1% встречаются отрицательные, то однопоточная её реализация в среднем отработает за 29 мс, в то время как на 128 потоках работа программы займёт уже 105 мс. Если вероятность ошибки составит 10%, то на 128 потоках программа будет работать 6425 мс.

Почему так? Дело в том, что библиотека `libunwind`, в общем-то, была не вполне рассчитана на такое. Не все ресурсы, необходимые для обработки исключений,

располагаются в потокобезопасных хранилищах, поэтому в нашем случае возникает чудовищная конкуренция, которая и выливается в то, во что выливается.

Ещё Страуструп предупреждал, что механизм исключений не даёт гарантии производительности: раскрутка стека — это крайне дорогостоящая операция, которую компилятор совершенно не умеет оптимизировать; вдобавок, она сопряжена с тяжеловесной логикой обработки DWARF-таблиц и динамическими аллокациями. Как с этим жить? Ну, смириться, в общем-то.

Хотя, вообще говоря, исключения не должны быть рассчитаны на такое количество вызовов. Ситуация, возникающая с вероятностью 10%, на самом деле никакая не исключительная, а самая обычная, а мы знаем, что очень плохо, когда исключения возникают в обычной логике программы. С этой точки зрения можно сказать, что если они настолько нагружают систему, то это совершенный архитектурный провал, и надо бы пересмотреть свою жизненную философию или ещё что-нибудь. А можно испугаться и вообще никогда больше не пользоваться исключениями. Выводы из приведенных бенчмарков следует сделать самостоятельно.

Тем не менее, проблема с обработкой исключений в многопоточных приложениях действительно имеет место быть, и во многом именно из этих соображений были предложены альтернативы, одна из которых будет рассмотрена далее.

`std::expected` (C++23)

Мотивация

Шаблонный класс `std::expected<T, E>` предоставляет интерфейс, отражающий единственное из двух значений: *ожидаемое* (*expected*) типа `T` или *неожиданное* (*unexpected*) типа `E`. Это современная альтернатива как исключениям, так и кодам возврата, попытавшаяся вобрать в себя преимущества обоих из них. Он был вдохновлён преимущественно функциональными языками, такими как Rust и Huskell.

Использование `std::expected` прозрачно с точки зрения видимости ошибки. Как и при работе с исключениями, полнота отладочной информации, предоставляемой `std::expected`, ограничивается лишь фантазией программиста. Код, порождаемый им, остаётся относительно чистым сравнительно кодов возврата; мы также оставляем при себе возможность делигировать обработку ошибок другому слою программы и, более того, можем использовать `std::expected` в качестве посредника между кодами возврата и исключениями. При этом не монополизирует канал связи, поскольку единственный `return` предназначен для возврата одновременно как корректного результата, так и

ошибки.
И, конечно, `std::expected` полностью лишён проблемы конкуренции между потоками.

Впрочем, по достаточно понятным причинам он не является полноценной заменой исключениям — это скорее инструмент для мгновенной обраотки локальных, ожидаемых ошибок.

Интерфейс

Конструктор `std::expected` по умолчанию принимает ожидаемое значение. Если мы хотим задать ошибку, то мы должны передать ему объект специального типа `std::unexpected`, название которого, в целом, в достаточной степени отражает его семантику:

```
std::expected<int, std::string> exp1(238);
std::expected<int, std::string> exp2(std::unexpected<std::string>
("Pupupu"));
```

При этом также имеют место параметры `std::in_place` и `std::unexpect`, которые конструируют аргументы in-place. `std::unexpect`, помимо прочего, предоставляет несколько более удобный интерфейс задания ошибочного состояния:

```
std::expected<int, std::string> exp2(std::unexpect, "Pupupu");
```

При копирующем присваивании ожидаемое значение задаётся значением своего же типа, а для задания ошибки необходимо использовать класс `std::unexpected<E>`:

```
std::expected<int, std::string> exp2 = std::unexpected<std::string>
("pupupu");
```

Для доступа к содержимому этого класса используются методы `.value()` и `.error()`. Их поведение весьма примечательно, поскольку если в момент обращения к `.value()` объект находится в невалидном состоянии, происходит... выброс исключения `bad_expected_access`. А поведение в случае невалидного обращения к `.error()` не определено.

Это может показаться странным: мы вроде бы хотели отказаться от исключений, а тут такое. Из этих соображений доступ к ожидаемому полю также можно получать при помощи операторов `*` и `->`, хотя в этом случае обращение к объекту в невалидном состоянии провоцирует неопределенное поведение. А определить, валидное ли оно,

можно при помощи метода `has_value` или, что бывает особенно удобно, преобразования в булево значение:

```
std::expected<int, std::string> exp(238);
std::cout << std::boolalpha << bool(exp) << std::endl;       // true
std::cout << exp.value() << std::endl;                       // 238
std::cout << *exp << std::endl;                              // 238
exp = std::unexpected<std::string>("pupupu");
std::cout << std::boolalpha << exp.has_value() << std::endl; // false
std::cout << exp.error() << std::endl;                       // pupupu
try {
    std::cout << exp.value() << std::endl;                    // Исключение
}
catch(...) {
    std::cout << "Caught an exception!\n";
}
```

Существуют также методы `value_or` / `error_or`, которые позволяют вернуть ожидаемое значение или ошибку соотвественно либо иной результат, если запрошенное значение отсутствует:

```
std::expected<int, std::string> exp(238);
std::cout << exp.value_or(0) << std::endl; // 238
std::cout << exp.error_or("OK") << std::endl; // OK
exp = std::unexpected<std::string>("pupupu");
std::cout << exp.value_or(0) << std::endl; // 0
std::cout << exp.error_or("OK") << std::endl; // pupupu
```

Монадические операции

Одной из особенностей `std::expected` является возможность модифицировать его значения, избегая проверки громоздких условий. Для этого используются четыре метода:

> ▶ `transform`
>
> Если объект содержит ожидаемое значение, применяет к нему вложенну функцию. Возвращает объект типа `std::expected`, который содержит либо результат выполнение функции, либо неожиданное значение исходного объекта
>
> ```
> int Square(int x) {
>  return x * x;
> }
> ```

```
int main() {
    std::expected<int, int> exp = 5;
    std::cout << exp.transform(Square).value() << std::endl; // 25
    exp = std::unexpected<int>(-1);
    std::cout << exp.transform(Square).error() << std::endl; // -1
}
```

▶ `transform_error`

Аналог `transform`, модифицирующий неожиданное значение.

```
int Square(int x) {
    return x * x;
}

int main() {
    std::expected<int, int> exp(std::unexpect, 5);
    std::cout << exp.transform_error(Square).error() << std::endl; // 25
    exp = -1;
    std::cout << exp.transform_error(Square).value() << std::endl; // -1
}
```

▶ `and_then`

Аналог `transform`, совместимый с функциями, возвращающими `std::expected`.

```
std::expected<int, int> Square(int x) {
    return x * x;
}

int main() {
    std::expected<int, int> exp = 5;
    std::cout << exp.and_then(Square).value() << std::endl; // 25
    exp = std::unexpected<int>(-1);
    std::cout << exp.and_then(Square).error() << std::endl; // -1
}
```

▶ `or_else`

Аналог `transform_error`, совместимый с функциями, возвращающими `std::expected`.

```
std::expected<int, int> Square(int x) {
    return x * x;
```

```
}

int main() {
    std::expected<int, int> exp(std::unexpect, 5);
    std::cout << exp.or_else(Square).error() << std::endl; // 25
    exp = -1;
    std::cout << exp.or_else(Square).value() << std::endl; // -1
}
```

Все эти методы позволяют функционально изменять объект, не прибегая к дополнительным проверкам.

N способов убить программу

Что совсем отлично, так это то, что теперь мы знаем по меньшей мере три способа завершить программу: `std::exit`, `std::abort` и `std::terminate`. А вообще их, на самом деле, по меньшей мере пять, но мы остановимся на этих трех и кратко поясним, чем они отличаются.

`std::exit` подразумевает *нормальный* выход из программы, то есть не аварийный, то есть почти такой же, как и при возвращении из `main`. Завершение при помощи `std::exit` полностью очищает все динамически захваченные ресурсы, но не раскручивает стек.

`std::abort` подразумевает как раз аварийный выход и никакие ресурсы не очищает. Он поднимает POSIX-флаг SIGABRT, который можно перехватить для ручной обработки.

`std::terminate` служит интерфейсом, специально введённым для обработки исключений. По умолчанию он выводит лог и вызывает `std::abort`, но его поведение можно изменить при помощи функции `std::set_terminate`:

```
void foo() {
    std::cout << "LizaWTF\n";
}

int main() {
    std::set_terminate(foo);
    throw 238;
}
```

Программа в любом случае аварийно завершится, если исключение не будет обработано, однако перед этим выполнится тело заданной пользователем функции.

Интересно, что похожего рода параметризация поддерживается и `std::exit`. Для этого используется функция `std::atexit`, позволяющая задать инструкции, которые будут выполняться при нормальном завершении программы:

```
void foo() {
    std::cout << "LizaWTF\n";
}

void faa() {
    std::cout << "faa()\n";
}

int main() {
    std::atexit(foo);
    std::atexit(faa);
}

// faa()
// LizaWTF
```

Источники

1. А. П. Хвастунов — Лекции по основам программирования на C++, 2 семестр, 2026
2. cppreference.com — Diagnostics library (https://en.cppreference.com/w/cpp/error.html)
3. cppreference.com — Exceptions (https://en.cppreference.com/w/cpp/language/exceptions.html)
4. @kahi4 — C++ exception handling под капотом или как же работают исключения в C++ (цикл переводов статей Николаса Браиловски) (https://habr.com/ru/articles/279111/)
5. Itanium C++ ABI (https://itanium-cxx-abi.github.io/cxx-abi/abi-eh.html#imp-catch)
6. Борис Муратшин — Что же там такого тяжелого в обработке исключений C++? (https://habr.com/ru/articles/208006/)
7. @honyaki — Альтернативы исключениям C++ и зачем они нужны (https://habr.com/ru/companies/skillfactory/articles/654723/)
8. Daniel Lemire — Avoid exception throwing in performance-sensitive code (https://lemire.me/blog/2022/05/13/avoid-exception-throwing-in-performance-sensitive-code/)
9. Proposal P2544R0 (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2544r0.html)
10. Proposal P0323R11 (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0323r11)

11. cppreference.com — `std::expected` (https://en.cppreference.com/w/cpp/utility/expected.html)

---

1. Так поступают некоторые стандартные функции с макросом `errno` (от *error number*), который разворачивается в потокобезопасную глобальную переменную и куда записывается один из множества POSIX-совместимых кодов возврата — для полного списка см. https://en.cppreference.com/w/cpp/error/errno_macros.html ↩
2. Для нежелающих верить — см. Issue 388 (https://cplusplus.github.io/CWG/issues/388.html) и пример в разделе Notes статьи про `catch` (https://en.cppreference.com/w/cpp/language/catch.html) ↩
3. ...потому что это ужасно, но для особенно любопытных в списке источников оставлена ссылка на перевод очень хорошей и глубокой статьи от Николаса Браиловски, которая крайне рекомендуется к прочтению. ↩